BHARATIYA VIDYA BHAVAN'S
SARDAR PATEL INSTITUTE OF TECHNOLOGY
Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai – 400058-India
Department of Computer Engineering

Name	**Soham Prajapati**
UID no.	**2024300186**
Experiment No.	**01**

Program 1a	
AIM:	To implement the various functions e.g. linear, non-linear, quadratic, exponential etc.
PROBLEM STATEMENT:	For this experiment, you have to implement at least 10 functions from the following list.
PROGRAM:	

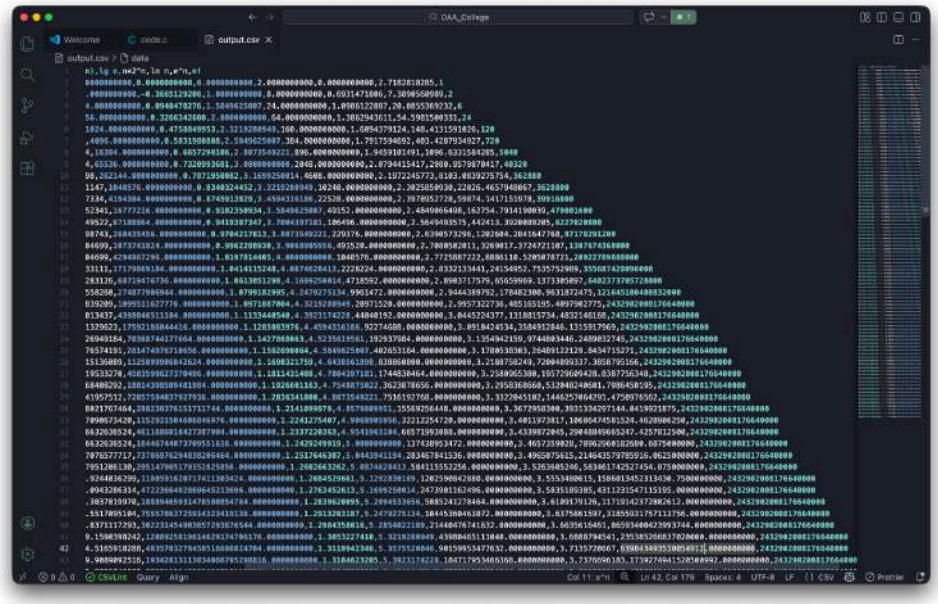
```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

/* the functions to implement are as follows:
   1. n
   2. (3/2)^n
   3. n^3
   4. (log base 2 (n))^2
   5. log base 2 (n!)
   6. 2^(2n)
   7. log(log(n))
   8. log base 2 (n)
   9. n * 2^n
   10. ln(n)
   11. e^n
   12. n! (factorial) - capped at 20 to avoid overflow
*/

// 1. n (identity)
int nValue(int n) {
   return n;
}

// 2. (3/2)^n
```

```
long double threeByTwo(int n) {
   return powl(3.0L/2.0L, (long double)n);
}

// 3. n^3
long long nCube(int n) {
   return (long long)n * n * n;
}

// 4. (log base 2 (n))
long double log2N(int n) {
   if (n <= 0) return 0.0L;
   return log2l((long double)n);
}

// 5. log base 2 (n!) computed as sum_{i=1..n} log2(i)
long double factorial(int n) {
   if (n <= 1) return 0.0L;
   int limit = n;
   long double logSum = 0.0L;
   for (int i = 2; i <= n; ++i) {
       logSum += log2l((long double)i);
   }
   return logSum;
}

long double log2OfFactorial(int n) {
   return factorial(n);
}

// 6. 2^(2n)
long double twoPow2n(int n) {
   return powl(2.0L, 2.0L * (long double)n);
}

// 7. log(log(n)) (natural logs)
long double logLogN(int n) {
   if (n <= 1) return 0.0L;
   return logl(logl((long double)n));
}
```

```
// 8. log base 2 (n) wrapper (already log2N)

// 9. n * 2^n
long double nTimes2PowN(int n) {
   return (long double)n * powl(2.0L, (long double)n);
}

// 10. natural log ln(n)
long double logN(int n) {
   if (n <= 0) return 0.0L;
   return logl((long double)n);
}

// 11. e^n
long double expN(int n) {
   return expl((long double)n);
}

long long factorialValue(int n) {
   if (n < 0) return 0; // undefined for negative
   if (n == 0 || n == 1) return 1;
   long long result = 1;
   for (int i = 2; i <= n && i <= 20; ++i) { // cap at 20 to avoid
overflow
       result *= i;
   }
   return result;
}

int main() {
   FILE *f = fopen("output.csv", "w");
   if (!f) {
       perror("fopen");
       return 1;
   }
   fprintf(f, "n,(3/2)^n,n^3,(lg n)^2,lg(n!),2^(2n),log(log n),lg
n,n*2^n,ln n,e^n,n!\n");
   for (int i = 1; i <= 100; ++i) {
       int n = i;
```

```
        long double v_threeByTwo = threeByTwo(i);
        long long v_nCube = nCube(i);
        long double v_log2n = log2N(i);
        long double v_log2n_sq = (v_log2n == 0.0L) ? 0.0L : v_log2n
* v_log2n;
        long double v_log2_fact = log2OfFactorial(i);
        long double v_twoPow2n = twoPow2n(i);
        long double v_loglogn = logLogN(i);
        long double v_lg = v_log2n;
        long double v_nTimes2n = nTimes2PowN(i);
        long double v_lnn = logN(i);
        long double v_expn = expN(i);
        long long v_fact = factorialValue(i);

        fprintf(f,
"%d,%.10Lf,%lld,%.10Lf,%.10Lf,%.10Lf,%.10Lf,%.10Lf,%.10Lf,%.
10Lf,%lld\n",
                n, v_threeByTwo, v_nCube, v_log2n_sq, v_log2_fact,
v_twoPow2n, v_loglogn, v_lg, v_nTimes2n, v_lnn, v_expn, v_fact);
    }
    fclose(f);
    printf("Wrote output to output.csv\n");
    return 0;
}
```

OUTPUT:	10 functions output



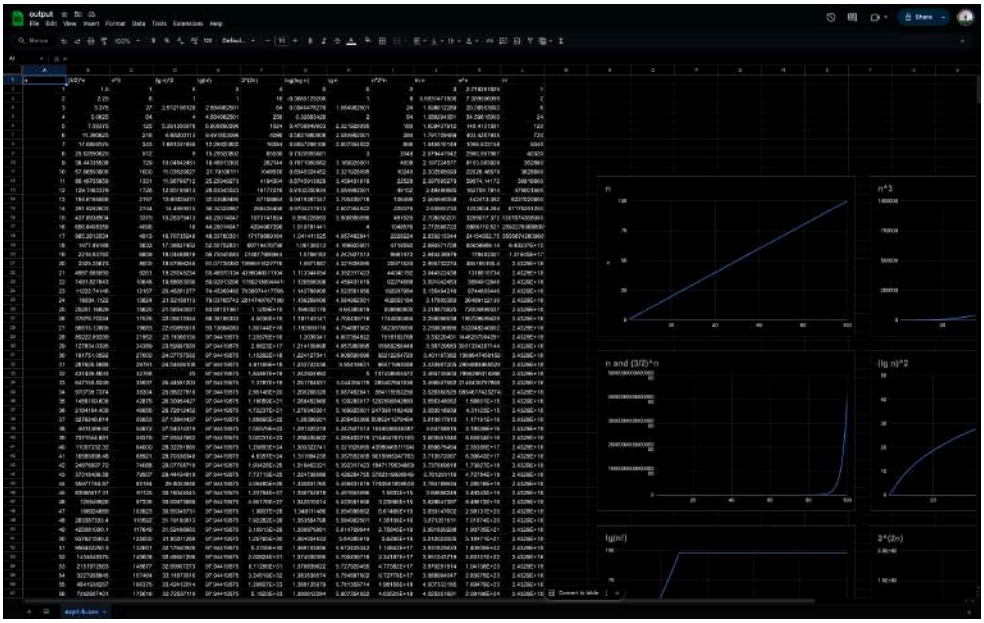
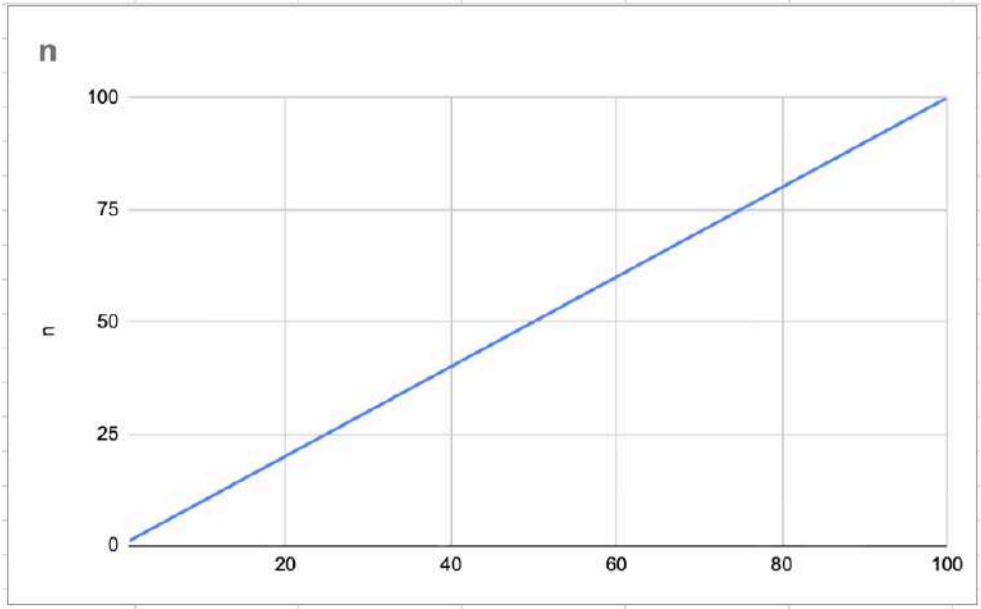
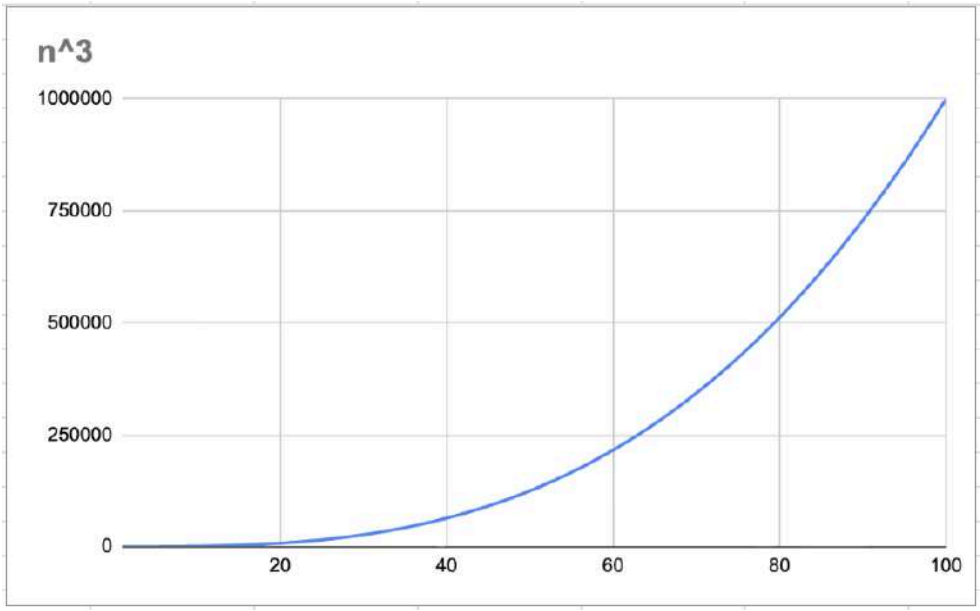
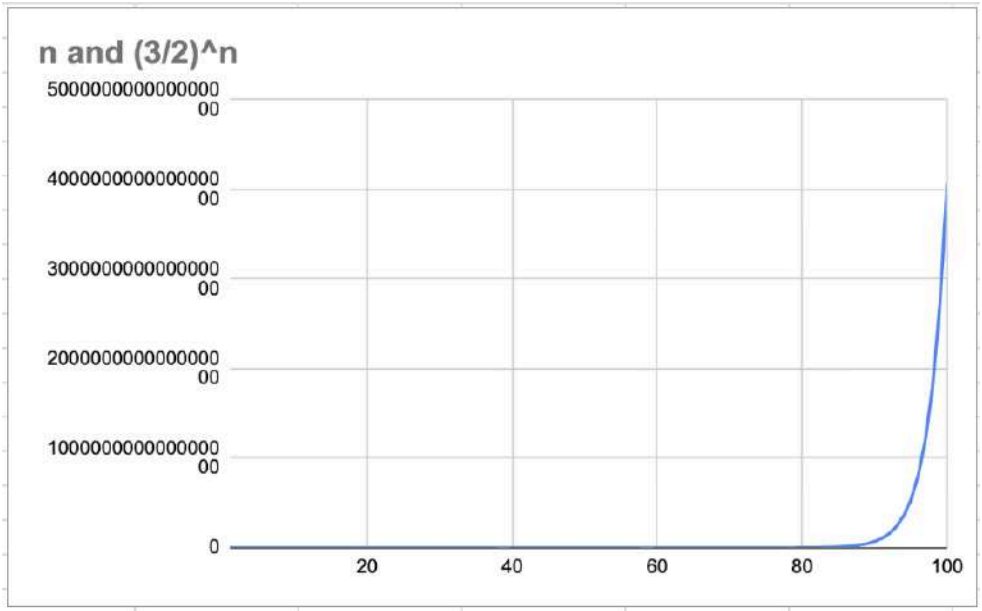
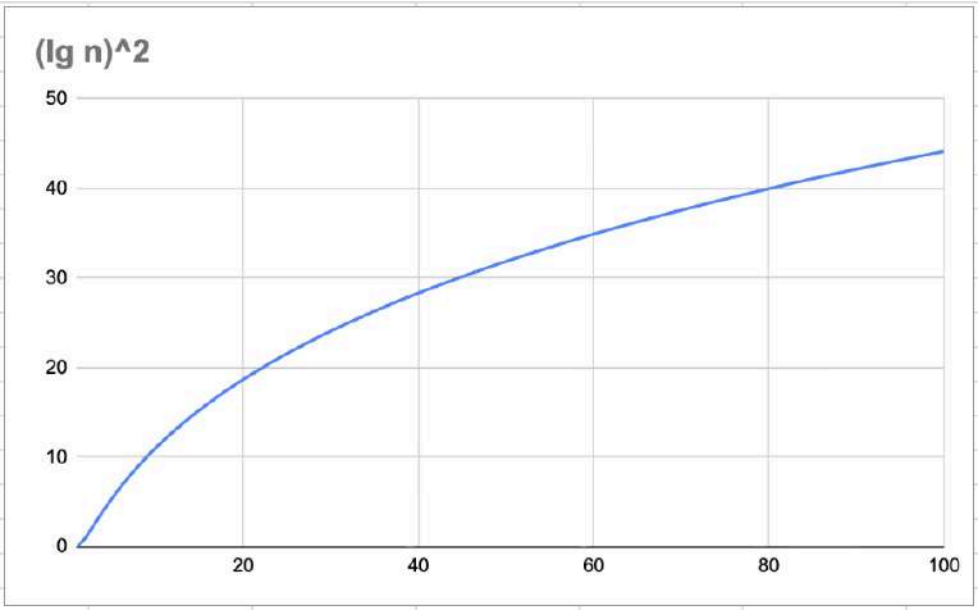
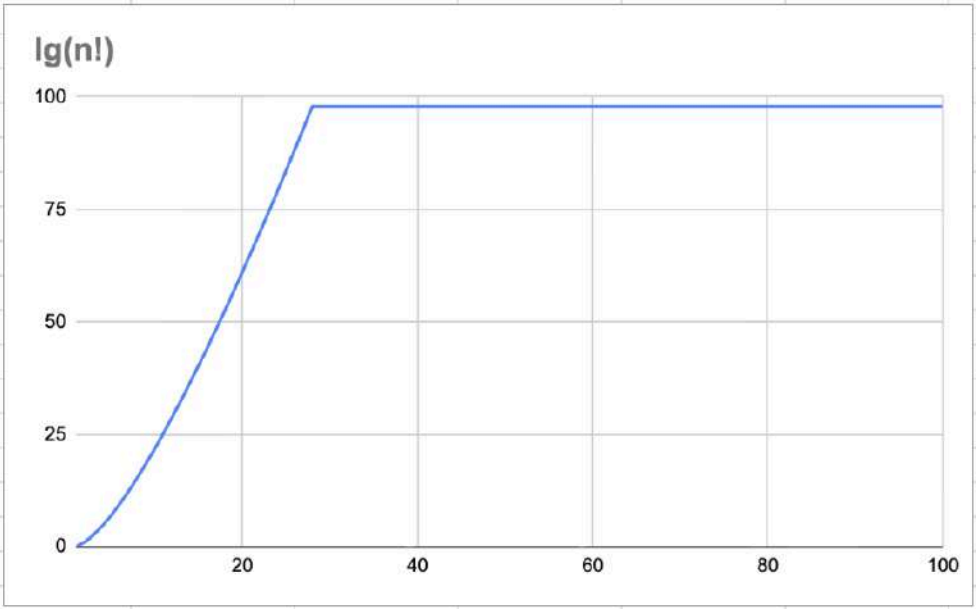
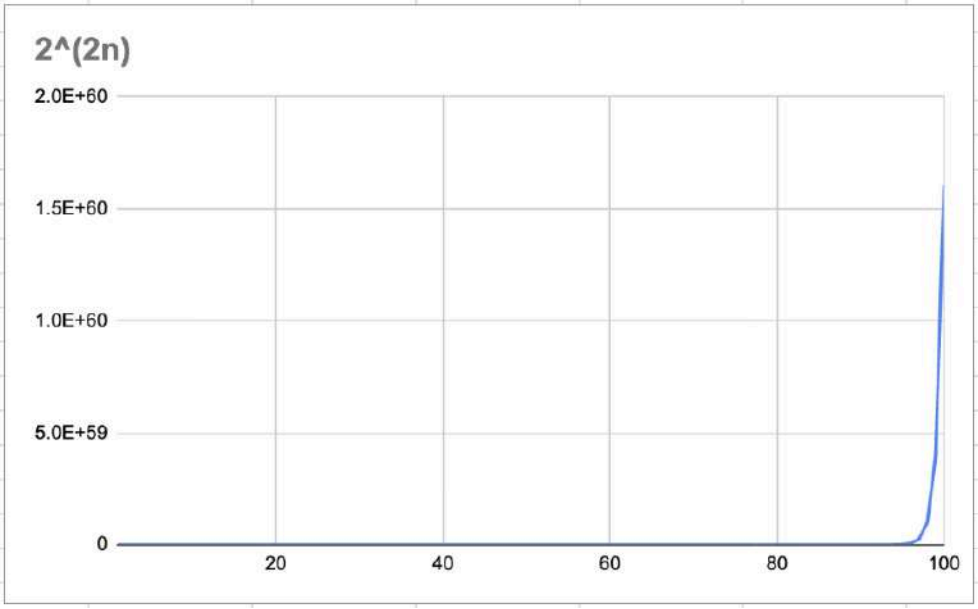


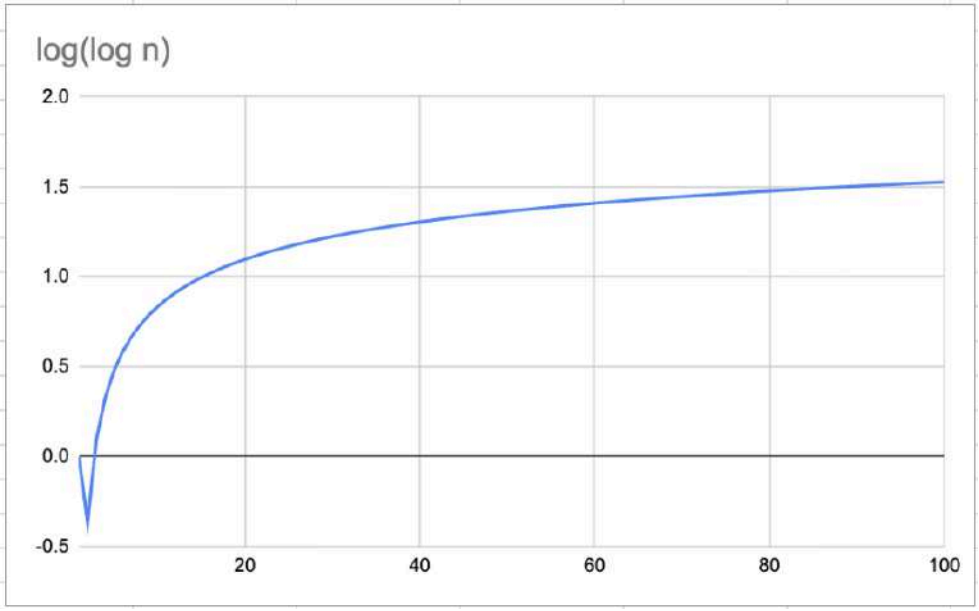
CHART:	 n 100 75 50 25 0 n 20 40 60 80 100  n^3 1000000 750000 500000 250000 0 20 40 60 80 100

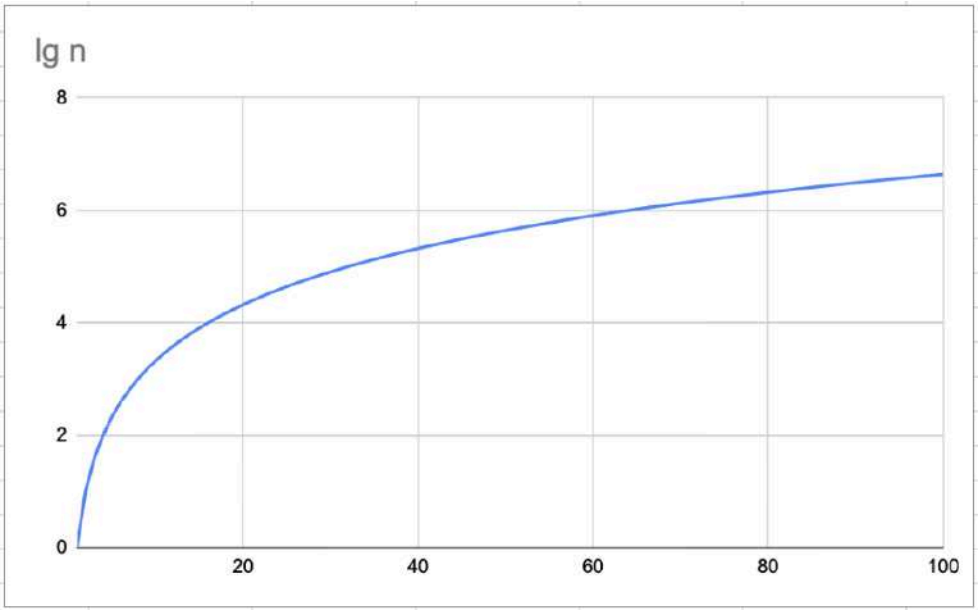

n and (3/2)^n
50000000000000000000
40000000000000000000
30000000000000000000
20000000000000000000
10000000000000000000
0
20
40
60
80
100

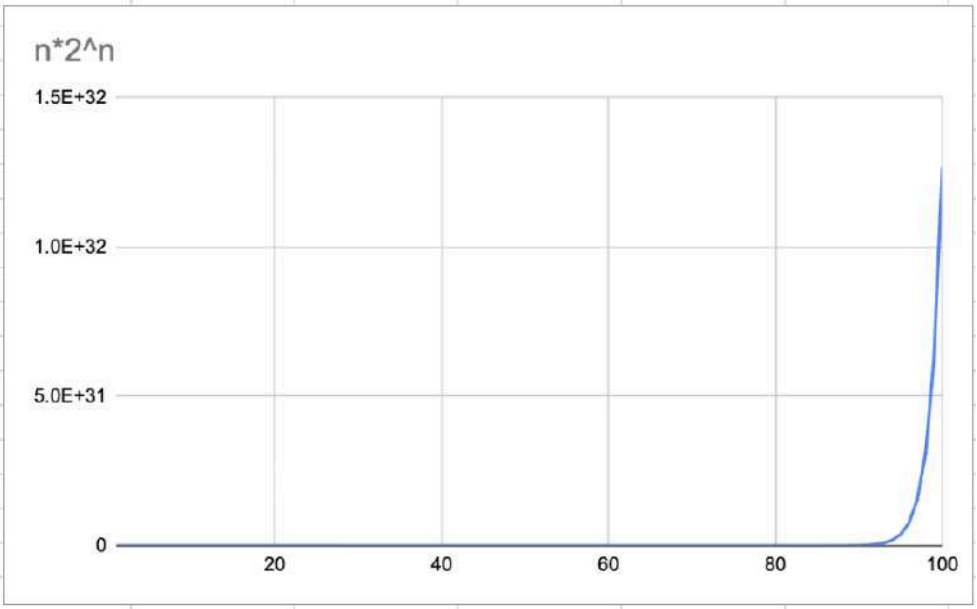

(lg n)^2
50
40
30
20
10
0
20
40
60
80
100

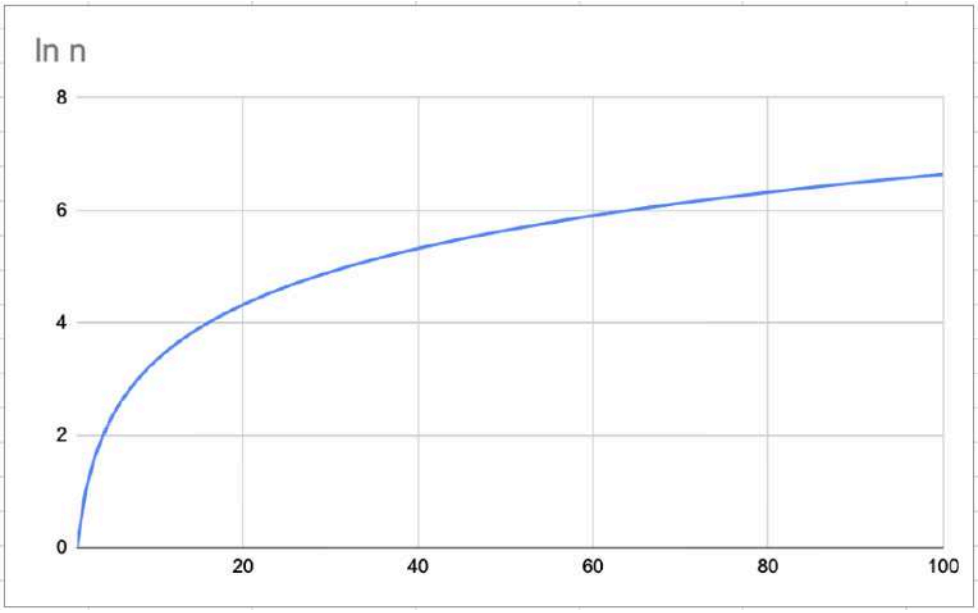

lg(n!)
100
75
50
25
0
20
40
60
80
100

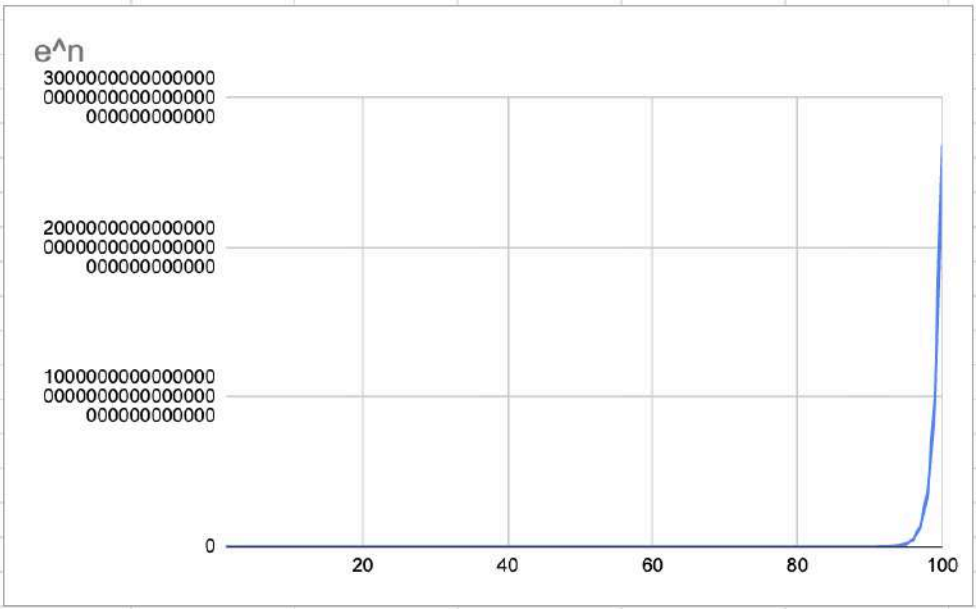

2^(2n)
2.0E+60
1.5E+60
1.0E+60
5.0E+59
0
20
40
60
80
100

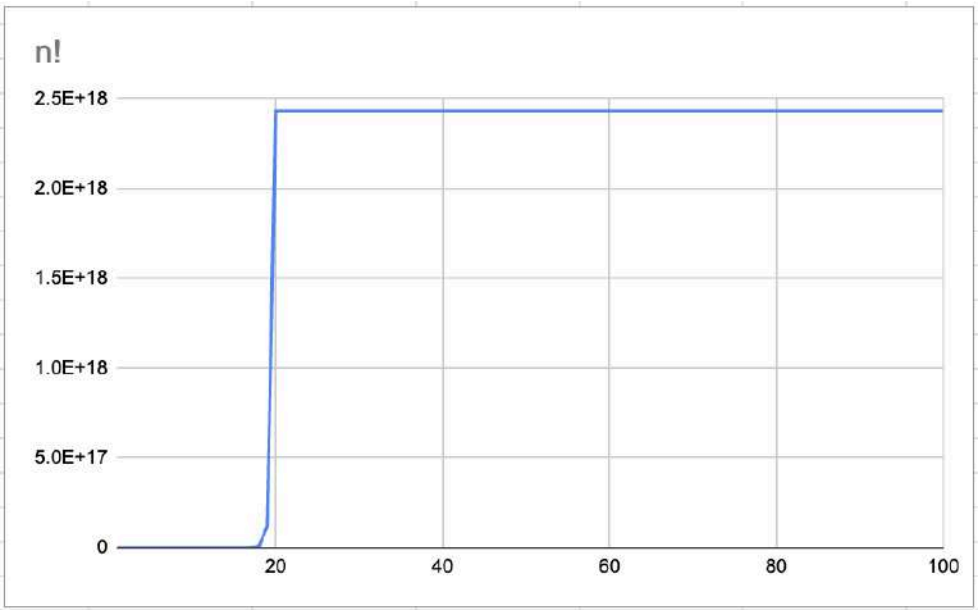

log(log n)
2.0
1.5
1.0
0.5
0.0
-0.5
20
40
60
80
100

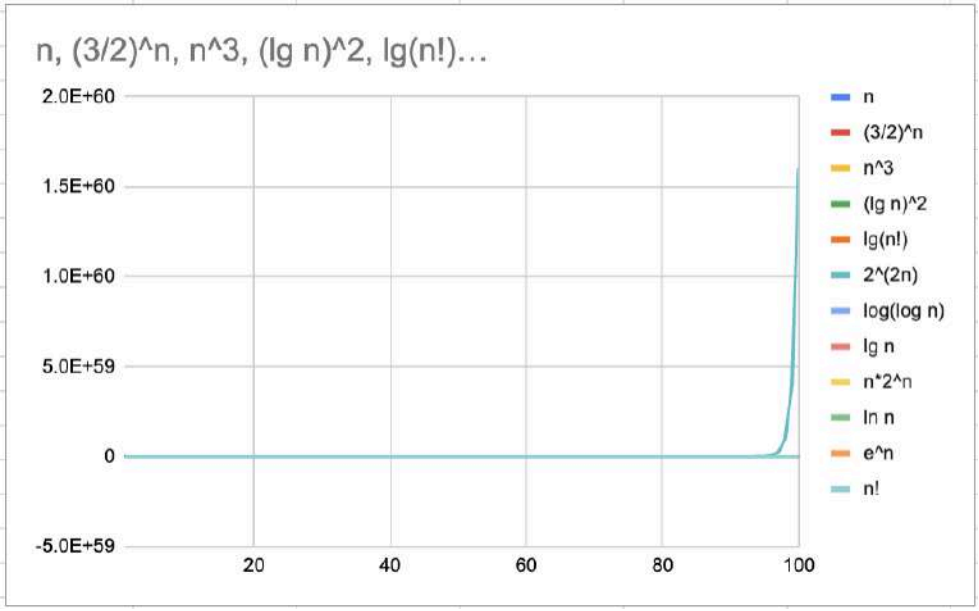

lg n
8
6
4
2
0
20
40
60
80
100


n*2^n
1.5E+32
1.0E+32
5.0E+31
0
20
40
60
80
100


ln n
8
6
4
2
0
20
40
60
80
100


e^n
3000000000000000
0000000000000000
000000000000
2000000000000000
0000000000000000
000000000000
1000000000000000
0000000000000000
000000000000
0
20
40
60
80
100


n!
2.5E+18
2.0E+18
1.5E+18
1.0E+18
5.0E+17
0
20
40
60
80
100

	
CONCLUSION:	The experiment provided a clear understanding of how various mathematical functions—such as linear, logarithmic, polynomial, and exponential—grow as input values increase. By calculating these functions for values of n ranging from 0 to 100, it became evident that each type follows a distinct growth pattern. Logarithmic functions increased the most slowly, followed by linear and polynomial functions with moderate growth. In contrast, exponential functions grew very rapidly compared to the others. The factorial function also exhibited extremely fast growth, so its evaluation was limited to values up to n = 20. The plotted graphs made these differences easy to visualize and highlighted how algorithm complexity scales with input size, allowing effective comparison of algorithm efficiency based on growth rates.
Program 1b	
AIM:	Experiment on finding the running time of an algorithm.
PROBLEM STATEMENT:	For this experiment, you need to implement two sorting algorithms namely Insertion and Selection sort methods. Compare these algorithms based on time and space complexity.
ALGORITHM:	Insertion Sort Algorithm:

```
INSERTION-SORT(A)
for j = 2 to A.length
    key = A[j]
    // Insert A[j] into the sorted sequence A[1 .. j − 1].
    i = j − 1
    while i > 0 and A[i] > key
        A[i + 1] = A[i]
        i = i − 1
    A[i + 1] = key
```

Selection Sort Algorithm:

SELECTION-SORT

```
for i = 1 to A.length − 1
    min_index = i
    for j = i + 1 to A.length
        if A[j] < A[min_index]
            min_index = j
    exchange A[i] with A[min_index]
```

PROGRAM:

10k Random Number Generator Code c file

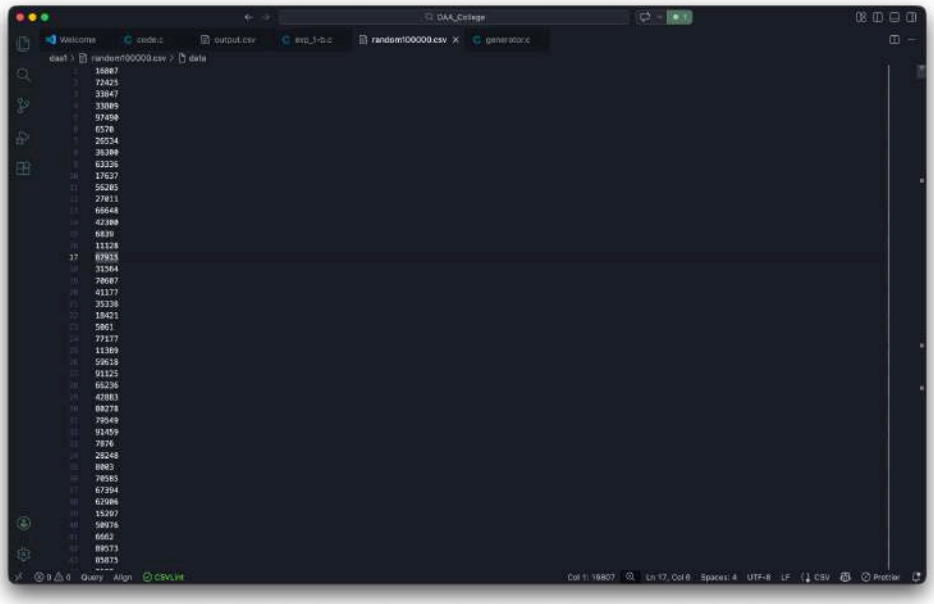
```c
#include <stdio.h>
#include <stdlib.h>

int main() {
   int N = 100000;
   FILE* fptr = fopen("random100000.csv", "a");
   for(int i = 0; i < N; i++) {
       int value = rand() % (N+1);
       fprintf(fptr, "%d\n", value);
   }
   fclose(fptr);
   return 0;
}
```

Insertion and Selection Sort c file

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void insertionSort(int arr[], int n) {
   for (int i = 1; i < n; i++) {
       int key = arr[i];
       int j = i - 1;
       while (j >= 0 && arr[j] > key) {
           arr[j + 1] = arr[j];
           j = j - 1;
       }
       arr[j + 1] = key;
   }
}

void selectionSort(int arr[], int n) {
   for (int i = 0; i < n - 1; i++) {
       int min_idx = i;
       for (int j = i + 1; j < n; j++)
           if (arr[j] < arr[min_idx])
               min_idx = j;
       int temp = arr[min_idx];
       arr[min_idx] = arr[i];
       arr[i] = temp;
   }
}

void copyArray(int src[], int dest[], int n){
   for(int i=0; i<n; i++){
       dest[i] = src[i];
   }
}

int main(void){
   FILE *fptr = fopen("random100000.csv", "r");
   if(fptr == NULL){
       printf("Error opening file.\n");
       return 1;
   }
```

```
    int originalArray[100000];
    for(int i=0; i<100000; i++){
        fscanf(fptr, "%d", &originalArray[i]);
    }
    fclose(fptr);

    int N = 1000;
    int m = 100;
    FILE *outptr = fopen("runningTimes.csv", "w");
    if(outptr == NULL){
        printf("Error opening output file.\n");
        return 1;
    }
    for(int i=m; i<=(N*m); i+=m){
        int arr1[100000];
        int arr2[100000];
        copyArray(originalArray, arr1, i);
        copyArray(originalArray, arr2, i);

        clock_t start1 = clock();
        insertionSort(arr1, i);
        clock_t end1 = clock();

        double time1 = (double)(end1 - start1) / CLOCKS_PER_SEC;

        clock_t start2 = clock();
        selectionSort(arr2, i);
        clock_t end2 = clock();

        double time2 = (double)(end2 - start2) / CLOCKS_PER_SEC;

        fprintf(outptr,"%d,%lf,%lf\n", i, time1, time2);
    }
    fclose(outptr);
    return 0;
}
```

OUTPUT:



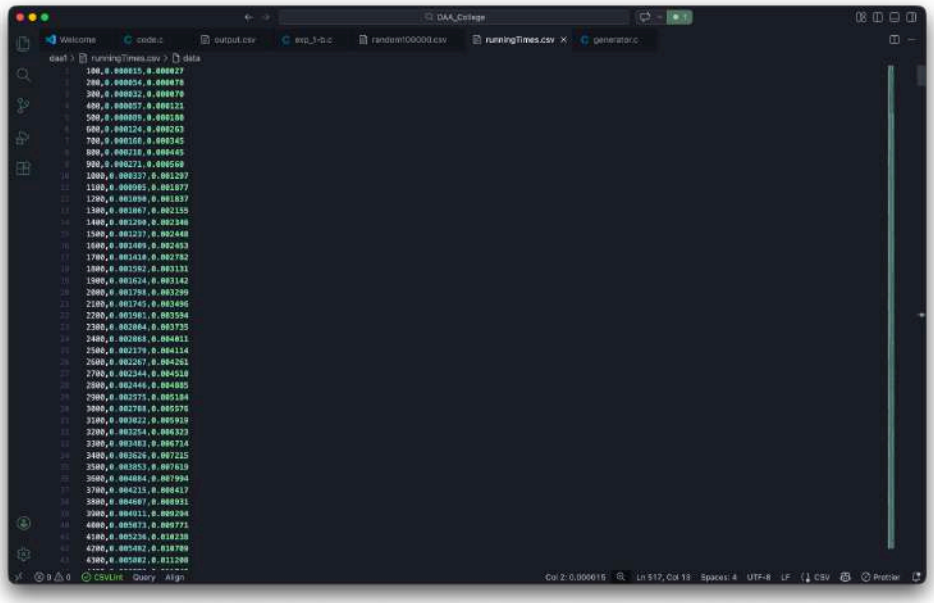
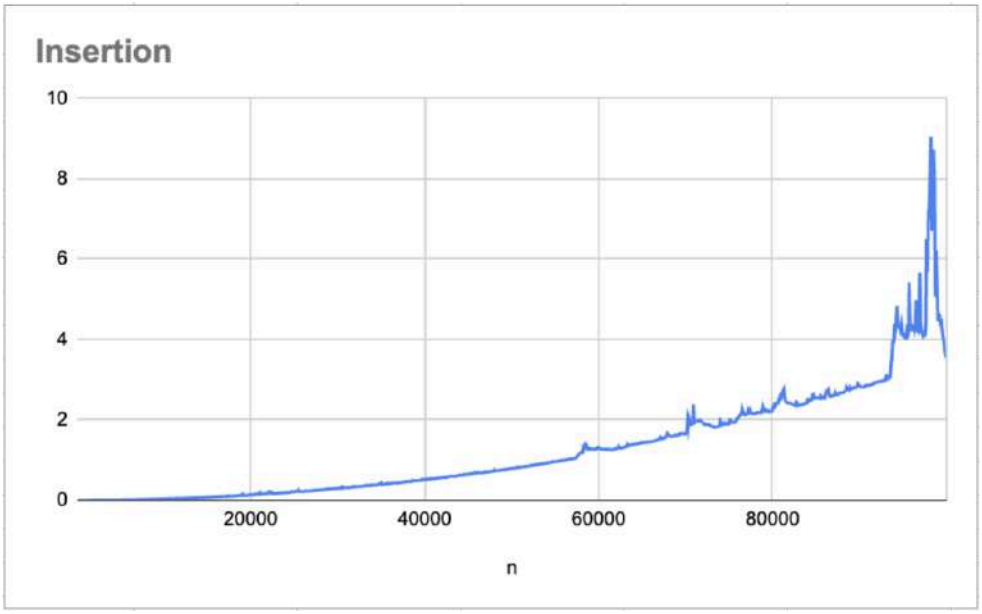
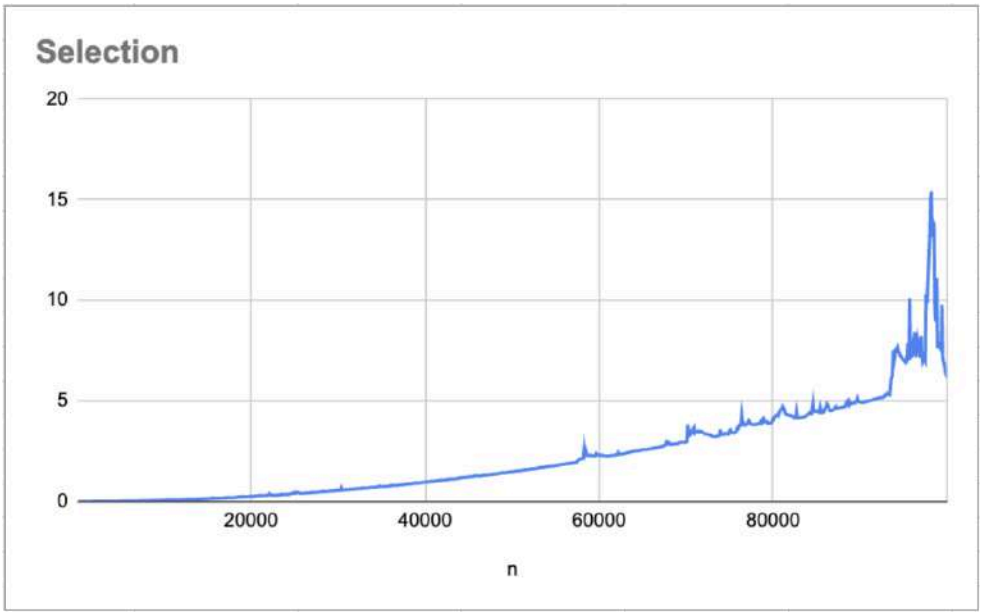
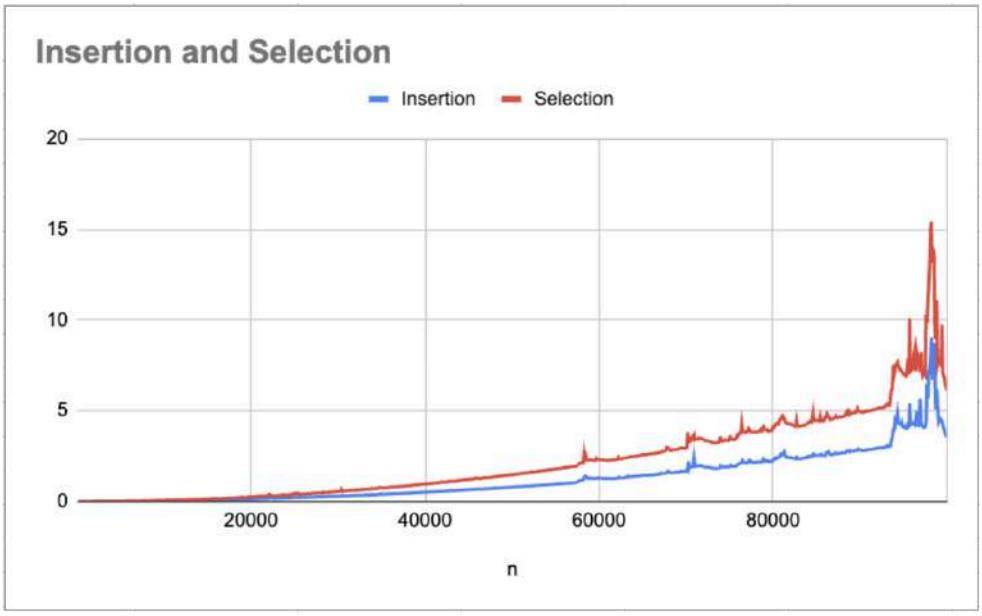


CHART:


Insertion
10
8
6
4
2
0
20000
40000
60000
80000
n


Selection
20
15
10
5
0
20000
40000
60000
80000
n

	
CONCLUSION:	From the comparison of insertion sort and selection sort, it can be concluded that both algorithms have a time complexity of $O(n^2)$ in the average and worst cases. However, insertion sort tends to run faster on partially sorted data because it requires fewer comparisons and movements. Selection sort, on the other hand, performs a fixed number of comparisons regardless of how the input is arranged, so it is not adaptive to existing order. With respect to space usage, both algorithms operate in-place and need only constant extra memory, giving them O(1) space complexity. As a result, insertion sort is usually a better choice for small or nearly sorted datasets, while selection sort is easier to understand but less efficient for larger inputs.